**Universidade de São Paulo**
**Escola de Artes, Ciências e Humanidades**
**Sistemas de Informação**

# Relatório do Exercício-Programa

ACH2034 - Organização e Arquitetura de Computadores I

Gustavo Neves Moraes — NUSP: 16986195
Vinícius Cunha Pereira — NUSP: 16973242

Profa. Gisele S. Craveiro

São Paulo
2026

# Sumário

**1 Organização e Arquitetura MIPS** **2**

**2 Descrição do problema e código alto nível da solução** **2**
2.1 Descrição do problema . . . . . 2
2.2 Algoritmo de solução . . . . . 2

**3 Código em Assembly desenvolvido** **4**
3.1 Bloco principal (`main`) . . . . . 4
3.2 Sub-rotina `mmc` . . . . . 5
3.3 Sub-rotina `divide` . . . . . 7

**4 Explicação detalhada das instruções utilizadas no código feito pela equipe** **8**

# 1 Organização e Arquitetura MIPS

Muito texto.

# 2 Descrição do problema e código alto nível da solução

## 2.1 Descrição do problema

O problema resolvido corresponde à Questão 8 da "Lista de Problemas"do EP.

O item (a) pede a implementação de uma função com assinatura `int divide (int *m, int *n, int d)`, que recebe três inteiros positivos e deve devolver 1 caso o divisor `d` divida pelo menos um dos valores apontados por `m` ou `n`, e 0 caso contrário. Além de retornar esses valores, sempre que `d` divide `*m`, o valor apontado é atualizado para o quociente da divisão, e o mesmo vale para `*n`.

O item (b) pede um programa que leia dois inteiros positivos `m` e `n` e, utilizando obrigatoriamente a função do item (a), calcule o mínimo múltiplo comum (MMC) entre eles.

## 2.2 Algoritmo de solução

A estratégia adotada para resolução do problema foi a de fatoração por tentativa. Testa-se sequencialmente cada divisor candidato `d`, começando em 2. Para cada `d`, chama-se `divide(&m, &n, d)`. Se a chamada indicar que pelo menos um dos dois valores foi dividido, o divisor `d` é incorporado ao acumulador do MMC (multiplicando-o) e o mesmo `d` é testado novamente, pois ele pode dividir novamente algum dos dois valores restantes. Caso `d` não divida nem `*m` nem `*n`, incrementa-se `d` e repete-se o processo. O algoritmo termina quando `m` e `n` foram reduzidos a 1, momento em que todos os seus fatores primos (com multiplicidade) já foram extraídos e incorporados ao acumulador, que corresponde, portanto, ao MMC entre os valores originais.

A seguir, o Código 1 apresenta uma versão em linguagem C do algoritmo, servindo de ponte entre a descrição acima e a implementação em Assembly apresentada na Seção **??**.

```
1 int divide(int *m, int *n, int d) {
2     int dividiu = 0;
3
4     if (*m % d == 0) {
```

```
        *m /= d;
        dividiu = 1;
    }
    if (*n % d == 0) {
        *n /= d;
        dividiu = 1;
    }

    return dividiu;
}

int MMC(int m, int n) {
    int result = 1;
    int d = 2;

    while (m != 1 || n != 1) {
        if (divide(&m, &n, d)) {
            result *= d;
        } else {
            d++;
        }
    }

    return result;
}

int main() {
    int m, n;
    scanf("%i", &m);
    scanf("%i", &n);

    int mmc = MMC(m, n);

    printf("O MMC e: %i\n", mmc);
    return 0;
}
```

Listing 1: Pseudocódigo em C do algoritmo

Vale comentar que uma possível otimização para a estratégia adotada seria testar apenas divisores primos, em vez de verificar todos os inteiros sequencialmente. Isso porque não faz sentido, testar divisores compostos, como por exemplo 6, já que seus fatores primos (2 e 3) necessariamente teriam sido testados anteriormente. Dessa forma, poderia ser mantido um vetor contendo números primos, e a variável `d` passaria a assumir apenas esses valores, reduzindo a quantidade de testes desnecessários e tornando o processo de fatoração mais eficiente, principalmente para entradas de maior magnitude.

# 3 Código em Assembly desenvolvido

O programa foi implementado em MIPS Assembly e é organizado em três sessões principais: a rotina principal (`main`), responsável pela leitura dos dados e impressão do resultado; a sub-rotina `mmc`, que implementa o laço de cálculo do Mínimo Múltiplo Comum por divisões sucessivas; e a sub-rotina `divide`, chamada repetidamente por `mmc`, que tenta dividir `m` e/ou `n` por um divisor candidato `d`.

A Tabela 1 resume o papel de cada registrador utilizado, seguindo a convenção de uso da arquitetura MIPS (registradores `$t` como temporários não preservados entre chamadas, `$s` como registradores salvos/persistentes, `$a` para argumentos e `$v` para valores de retorno).

| Registrador | Papel no programa |
|---|---|
| `$sp` | Ponteiro de pilha, usado para empilhar parâmetros e contexto salvo |
| `$ra` | Endereço de retorno, salvo/restaurado na pilha em `mmc` e `divide` |
| `$s0` | Acumulador do MMC (salvo na pilha durante as chamadas a `divide`) |
| `$s1` | Divisor candidato `d` (salvo na pilha durante as chamadas a `divide`) |
| `$a0, $a1, $a2` | Parâmetros de `divide` (&m, &n, d) após serem lidos na pilha |
| `$v0` | Código de *syscall* / valor de retorno de `divide` (0 ou 1) |
| `$t0, $t1, $t2` | Temporários intermediários (dividendo, resto, quociente) |

Tabela 1: Registradores utilizados e seus papéis

## 3.1 Bloco principal (`main`)

O bloco `main` realiza a leitura dos dois inteiros `m` e `n` via chamadas de sistema, chama a sub-rotina `mmc` (sem parâmetros explícitos, já que `mmc` acessa `m` e `n` diretamente na área de dados) e imprime o valor retornado em `$v0`.

```
1 main:
2         # Imprime string
3         li $v0, 4
4         la $a0, msg_m
5         syscall
6
```

```
7         # Le inteiro m
8         li $v0, 5
9         syscall
10        sw $v0, m
11
12        # Imprime string
13        li $v0, 4
14        la $a0, msg_n
15        syscall
16
17        # Ler inteiro n
18        li $v0, 5
19        syscall
20        sw $v0, n
21
22        # Chama a rotina mmc
23        jal mmc
24
25        # Salva o retorno (o MMC calculado) em $t0
              temporariamente
26        move $t0, $v0
27
28        # imprime string de msg MMC
29        li $v0, 4
30        la $a0, msg_mmc
31        syscall
32
33        # imprime string do MMC
34        li $v0, 1
35        move $a0, $t0
36        syscall
37
38        # acaba o programa ;D
39        li $v0, 10
40        syscall
```

Listing 2: Bloco `main`

## 3.2 Sub-rotina `mmc`

A sub-rotina `mmc` inicia salvando na pilha o endereço de retorno (`$ra`) e os registradores `$s0` e `$s1`, que precisam sobreviver às chamadas a `divide`. Em seguida executa o laço principal do algoritmo: a cada iteração verifica se `m` e `n` já foram reduzidos a 1 (rótulo `fim_mmc`); caso contrário, empilha os três parâmetros de `divide` (&m, &n e o divisor candidato `d`, nessa ordem) e realiza a chamada com `jal`. Se pelo menos um dos dois valores foi dividido, o acumulador `$s0` é multiplicado por `d` e o laço reinicia com o mesmo `d`; caso

contrário, `d` é incrementado (rótulo `nao_dividiu`). Ao final, o resultado é movido para `$v0`, o contexto salvo é restaurado e a sub-rotina retorna com `jr $ra`.

```
mmc:
        # Prologo da funcao mmc: salva o contexto na pilha
        addi $sp, $sp, -12
        sw   $ra, 8($sp)  # Salva endereco de retorno
        sw   $s0, 4($sp)  # Salva $s0
        sw   $s1, 0($sp)  # Salva $s1

        li $s0, 1 # $s0 = mmc = 1
        li $s1, 2 # $s1 = d = 2

mmc_loop:
        # carrega m e n
        lw $t0, m # $t0 = m
        lw $t1, n # $t1 = n

        # $t2 = minimoMMC = 1
        li $t2, 1

        # se m!=1 continua tentando dividir
        bne $t0, $t2, tentaDividir

        # A essa ponto do codigo pelo menos m eh 1
        # mas talvez n ainda pode ser divisivel

        # se n!=1 continua tentando dividir
        bne $t1, $t2, tentaDividir

        j fim_mmc

tentaDividir:
        # ----------------------------------------------------
        # Params
        #   offset 8($sp) -> &m   (endereço de m)
        #   offset 4($sp) -> &n   (endereço de n)
        #   offset 0($sp) -> d    (valor do divisor testado)
        # Quem chama abre espaço na pilha, empilha os 3
            parâmetros, faz o
        # "jal" e, ao retornar, desempilha o espaço que abriu.
        # ----------------------------------------------------
        addi $sp, $sp, -12   # abre espaço para 3 palavras na
            pilha
        la   $t3, m
        sw   $t3, 8($sp)      # empilha &m
        la   $t3, n
        sw   $t3, 4($sp)      # empilha &n
```

```
44          sw   $s1, 0($sp)      # empilha d
45
46          jal divide
47
48          addi $sp, $sp, 12     # desempilha os parâmetros
49          beqz $v0, nao_dividiu # se não dividiu ninguém, tenta
                outro d.
50
51      mul $s0, $s0, $s1 # c.c. pelo menos um deles foi dividido.
            Multiplicar o MMC pelo d atual.
52          j mmc_loop
53
54  nao_dividiu:
55          addi $s1, $s1, 1 # d++
56          j mmc_loop
57
58  fim_mmc:
59          # Prepara retorno da funcao
60          move $v0, $s0     # Coloca o resultado (MMC) em $v0
61
62          # Epilogo: restaura o contexto
63          lw   $s1, 0($sp)
64          lw   $s0, 4($sp)
65          lw   $ra, 8($sp)
66          addi $sp, $sp, 12
67          jr   $ra          # Retorna para main
```

Listing 3: Sub-rotina `mmc`

## 3.3 Sub-rotina `divide`

A sub-rotina `divide` recebe seus três parâmetros pela pilha, em vez de registradores de argumento. Como a própria sub-rotina abre seu próprio espaço de pilha (16 bytes, para salvar `$ra`, `$t0`, `$t1` e `$t2`) antes de acessar os parâmetros empilhados pelo chamador, os deslocamentos usados para lê-los (`24($sp)`, `20($sp)` e `16($sp)`) já contabilizam esse deslocamento adicional. Após copiar os parâmetros para `$a0`, `$a1` e `$a2`, a lógica de divisão é a mesma de antes: testa `*m` e, independentemente do resultado, testa `*n`, retornando em `$v0` se pelo menos uma das duas divisões ocorreu.

```
1  divide:
2          addi $sp, $sp, -16
3          sw   $ra, 12($sp)   # salva endereço de retorno
4          sw   $t0, 8($sp)    # salva $t0
5          sw   $t1, 4($sp)    # salva $t1
6          sw   $t2, 0($sp)    # salva $t2
7
```

```
8           lw   $a0, 24($sp)    # $a0 = &m
9           lw   $a1, 20($sp)    # $a1 = &n
10          lw   $a2, 16($sp)    # $a2 = d
11
12          li $v0, 0 # retorno eh zero ate que se prove o contrario
13
14          # testa *m
15          lw $t0, 0($a0) # $t0 = *m
16          div $t0, $a2
17          mfhi $t1 # resto
18
19          bnez $t1, testa_n # se resto != 0, não divide
20
21          mflo $t2 # quociente
22          sw $t2, 0($a0) # *m = *m / d
23          li $v0, 1  # retorna 1 pois dividiu
24
25  testa_n:
26          lw $t0, 0($a1) # $t0 = *n
27          div $t0, $a2
28          mfhi $t1 # resto
29
30          bnez $t1, fim_divide # se resto != 0, termina
31
32          mflo $t2 # quociente
33          sw $t2, 0($a1) # *n = *n / d
34          li $v0, 1 # dividiu
35
36  fim_divide:
37          lw   $t2, 0($sp)
38          lw   $t1, 4($sp)
39          lw   $t0, 8($sp)
40          lw   $ra, 12($sp)
41          addi $sp, $sp, 16
42          jr $ra
```

Listing 4: Sub-rotina `divide`

# 4 Explicação detalhada das instruções utilizadas no código feito pela equipe

Muito texto.